§15 · The Living Brain — from equations to a life

A new chapter for "The Learning Brain as a system of equations." §14 turned the model on for four static experiments. This chapter is the living system: one standalone brain, born from a frozen teacher, that then thinks continuously, learns by talking to a stronger model and by seeing the web, on a wake/sleep rhythm it runs itself, growing over a lifetime, with a sense of time — inside fixed compute and memory budgets, watchable and interactive. It is built to be faithful FIRST: five different coupled structures, exactly as §1–§5 describe, not one monolithic network — four of them growable spiking networks (cortex, cerebellum, hippocampus, and the basal-ganglia medium-spiny actor-critic), bound by a §5 neuromodulatory glue (a diffuse scalar tone that gates their plasticity — modulator, not network). Every mechanism is the field guide's; this chapter is the implementation and the honest ledger.

Code: `sapience/brain/*.py`, `run_life.py`, `interface/{dashboard,tui}.py`. Runs on CPU or GPU.

15.1 Five spiking modules, not one network

The guide is emphatic (§11) that the brain is five *different* structures coupled by activation but cut off from each other's gradients (§6). The implementation honours that — each module is its own structure on a shared LIF substrate (`brain/spiking.py`), and each uses the architecture that best mimics *its* region. Four are genuinely spiking and growable; §5 is the neuromodulatory glue — a scalar tone, not a network:

module	structure (region-specific)	teaching signal	file
§3 **Cortex** (spiking, growable)	stacked leaky-integrate-and-fire recurrent layers; **readout taps the analog membrane** of the top layer, not the binary spike	next-byte error via **e-prop** (forward-in-time eligibility + top-down signal + 3-factor gate; §15.17); BPTT (= $\beta \to 0$ PC) opt-in	`spiking_brain.py`
§1 **Cerebellum** (spiking, growable)	sparse LIF granule expansion → Purkinje readout, with a **Golgi feedback-inhibition loop** holding granule sparsity at target	climbing-fibre error, delta rule $\Delta W = -\eta \cdot c \cdot g$	`spiking_modules.py`
§2 **Basal ganglia** (spiking, growable)	LIF **medium-spiny neurons** → critic + softmax actor; the dopamine RPE trains *both* (policy gradient)	dopamine RPE $\delta = r + \gamma V' - V$	`spiking_modules.py`
§4 **Hippocampus** (spiking, growable)	sparse dentate-gyrus separation + a modern-Hopfield store with **spiking CA3 winner-take-all** recall (LIF memory neurons + lateral inhibition → CA1 read-out)	one-shot, novelty-gated write; pattern completion	`spiking_modules.py`
§5 **Neuromod. glue** (scalar)	ACh/NE/DA/5HT tone as the three-factor gate M(t); the **ACh tone scales the cortex learning rate** per phase	$\Delta w = \eta \cdot M(t) \cdot e$	`spiking_modules.py`

The cortex is the deep learner that carries language; the cerebellum corrects, the hippocampus stores/recalls, the basal ganglia supplies reward/curiosity, the neuromodulators gate plasticity and set the wake/sleep tone (§7.2). Spikes and reward scalars cross the boundaries; gradients do not (§6). Unlike an earlier version where these four were built but **orphaned** (only the cortex ran), they are now wired into the living loop (§15.14) so the brain actually *operates* as five systems.

15.2 The spiking substrate

`brain/spiking.py`. Leaky integrate-and-fire neurons: `v ← β·v·(1−s) + Wx + Rs`, fire when `v > θ`. The Heaviside spike is non-differentiable, so the *pseudo-derivative* used by both learning rules is a **surrogate gradient** (fast-sigmoid). By default the cortex learns by **e-prop** (§15.17) — forward-in-time, local, no backward pass; the opt-in `learn_rule="bptt"` reference is surrogate-BPTT, which §3.5 identifies as predictive coding in the $\beta \to 0$ limit, so even the non-plausible

reference stays inside the paper's framework while the architecture is genuinely spiking (membranes, thresholds, spikes). The cortex's `TwoCompartmentLIF` implements §3.7 directly: a somatic compartment (basal feedforward + recurrent drive) and an apical dendrite carrying the top-down error that modulates firing — the substrate the §15.17 dendritic/burst-error toggle uses.

Membrane readout (a measured 2× win). The original cortex applied its readout head to the binary *spike* vector — ~1 bit/neuron, discarding the analog membrane the neuron actually integrates. A controlled A/B (wikitext-2, identical seed/data/budget) showed that reading the graded top-layer **membrane** instead nearly doubles next-byte accuracy (0.17 → 0.34) and drops bits/byte 4.48 → 3.39 — at **zero** faithfulness cost, because the internal computation (inter-layer signals, recurrence) stays entirely spike-based; only the final observation taps the graded population code (as downstream cortex reads rates/LFP, not single spikes). This is now the default. A spike/tanh(membrane) *mix* and a squashed membrane were also tried and are no better; the plain membrane wins.

Two documented, off-by-default cortex levers. (i) `ALIFCell` — an adaptive-threshold LIF (spike-frequency adaptation, the LSNN long-timescale working memory). It is built and correct, but a four-arm A/B found it *loses* at the short byte-context this model uses (seq 48–128), where there is no long-range dependency for its slow state to exploit; it is kept for future long-horizon tasks. (ii) The §3.7 two-compartment top-down apical: faithful hierarchical top-down prediction requires stepping time-outer (layer-within-timestep), which would forfeit the vectorised speed-up below; since the membrane readout already supplies the graded-potential benefit, the two-compartment cell remains in the substrate for the §3.7 exposition but is not the running default. Both are honest trade-offs, not omissions.

A ~7× speed-up, bit-identical. The per-timestep Python loop originally recomputed the input projection `Win(x)` and the readout head every step. Because a feedforward LIF stack has no within-timestep top-down, the layers can run one-at-a-time over the whole sequence (`LIFCell.run_seq`): the input projection is one matmul over all T, the head is one matmul over all T, and only the genuinely-recurrent `Wrec(s)` stays in the loop. This is *mathematically identical* to stepping (verified max-diff 0.0) and cuts a training call from ~1.6–3.4 s to ~0.24 s. Growth stays identity-preserving under it.

15.3 Development: fixed neurons, evolving synapses (§10)

Development follows the biology: **the neuron count is largely set at birth** (neurogenesis is ~complete), so it is a *fixed, settable population* — small for a laptop, or scaled up toward the 100k–1M range the field guide imagines. What **evolves over the lifetime is the synaptic connectome**. The brain is born with a sparse connectome (a settable `syn_density`, default 0.5 of possible connections active); each night of the **child** phase runs **synaptogenesis** — `grow_synapses` activates a fraction of the currently-silent connections with fresh small weights, densifying the wiring — and each night of the **adolescent** phase runs **synaptic pruning** —

`prune` silences the weakest active connections, held at zero by a persistent mask so the pruning sticks. The **adult** does slow turnover. So the *synapse count* rises then settles while the *neuron count stays fixed*, exactly as synaptic density peaks in childhood and is pruned in adolescence — all riding the critical-period learning-rate envelope η(t)=η_max/(1+t/t_c) (§10.4).

Neurons can still be grown *deliberately* — `grow_neurons(add)` inserts LIF units whose *outgoing* weights are zero, so the network computes the same function the instant capacity appears (verified: post-grow bits/byte identical to pre-grow), and the synapse mask is padded identity-safely. This is the explicit lever (via `/api/arch`, or a large developmental step) to enlarge the population when there is real experience to justify it — capped at `max_model_gb`. Every part of the architecture (cortex, cerebellum, basal ganglia, hippocampus) reports its live **neuron and synapse counts** (`/api/arch`), and all of it — grow/prune synapses, grow neurons, re-seed a connectome — is editable live per part by API.

15.4 The capability fork, chosen honestly

The design study established a hard truth: a purely local-rule model over raw bytes learns byte statistics, not language. Two faithful ways forward, both grounded in §3.5: - **Spiking, five-module, growable (chosen — fidelity first):** the architecture above, learning by default via **faithful e-prop** (§15.17). Five distinct structures, four genuinely spiking + growable (cortex, cerebellum, hippocampus, basal ganglia) plus the §5 neuromodulatory glue. With the membrane readout and a longer context window its capability has improved from the original ~4 bits/byte to **~3.1–3.2 bits/byte** (born at ~3.3, evolving down over its first nights), with real word-like fragments emerging (*"…was from the … to the … and …"*). Still modest versus a rate GRU — the accepted cost of fidelity — but markedly better than before, and it keeps improving as it lives. - **Rate recurrent cortex (kept as the capability reference):** a byte GRU trained by plain backprop (= β→0 PC). It reaches ~0.5 bits/byte and writes coherent sentences (*"The ocean is … He also added with the starter Cullen …"*), but it is not spiking and cannot grow cleanly. Available as `--core rnn` for when capability is the priority. `brain/rnn_brain.py`.

The default is the spiking core; the fork is a one-line switch, and the paper states plainly which fidelity/fluency point each occupies.

15.5 Birth — the distillation kick-in

`life.birth`. Before it lives, the cortex is trained to competence on a **frozen Qwen3.5-0.8B** teacher's generated outputs (`brain/llm_teacher.py`) plus real text (wikitext-2) — the innate bootstrap. (This birth teacher is distinct from the in-*life* teacher it later converses with, Claude Sonnet 5 via `brain/partner.py`; the UI even labels the two, `teacher_name` flipping Qwen→Sonnet at birth's end.) It is *born able to write* (the rate cortex reaches coherent English at birth; the spiking cortex reaches its modest ceiling faster), then the life evolves it. This front-loads

the "fluency needs wall-time" cost into a one-time developmental window, as a genome + rapid early learning do. Birth stops adaptively — on the target bits/byte, OR on a **learning plateau** (no >0.01 bits/byte gain for six epochs), OR a wall-clock cap — because the spiking core floors near ~3 bits/byte and never reaches the rate-cortex target, so without the plateau stop it would burn every epoch; this cut birth from ~220 s to ~150 s.

15.6 One code for every sense

`brain/senses.py`. Sight, language, time and the brain's own voice become one stream of byte-levels 0–255 ("electricity"), each frame tagged by a "nerve" marker — the unified sensorimotor code. Motor output (`brain/motor.py`) is the same code in reverse: writing/ speaking = emitting bytes decoded to text; navigating = emitting a command. Vision enters via a headless browser (`brain/visual_web.py`) that navigates, screenshots and SCROLLS ("reads by scrolling"); its *readable text* is what the language cortex learns, its screenshot is shown as ASCII — raw pixels are not force-fed into the language mind-state (that would be noise).

15.7 The internal clock (a sense of time)

`senses.Clock`. A pacemaker-accumulator plus banks of oscillators at many periods (the striatal beat-frequency model + circadian rhythm) encodes elapsed time to byte-levels and STAMPS every perception — the fastest of §0's clocks, made a sensory channel. The brain always knows what time it is; a fixed time-probe measures how well it has learned to tell it.

15.8 Autonomous wake/sleep, and sleep that learns

`brain/life.py`. Sleep debt (§7.6) rises as the brain learns while awake; the brain DECIDES when to sleep — at least `min_awake`, by `max_awake` at the latest, and in between once the debt crosses a threshold. Sleep is the deep-learning phase, as in biology: it replays the whole-life memory heavily under the default learning rule (faithful e-prop, §15.17; waking thought stays fast while sleep does the consolidation) (§15.10), then applies a §8.4 SHY multiplicative downscale so it also renormalises rather than only potentiating. (Honestly this is an *open-loop* constant downscale — a small fixed factor over all weights — not yet a closed-loop restore of the §9 balance ⟨Δw⟩_wake + ⟨Δw⟩_sleep ≈ 0 scaled to the night's actual potentiation; that, and a protection term for replayed synapses, are named as the next refinement.) Moving deep consolidation into sleep more than doubled the learning rate while keeping waking thought fast (~9 thoughts/s). Each night the brain develops (§15.3).

15.9 Continuous thinking

The brain is never idle: a fast inner-monologue loop generates the next fragment of thought from the persistent spiking mind-state and streams it; a background sense-thread fetches Claude's teaching and web pages without ever pausing the thinking; your typed input is **non-blocking feedback** woven into the same stream (weighted stronger than passive input — more learning steps, tagged "you say:"), with in-band commands (`browse <url>` / `look <url>` redirect vision, `?time` queries the clock); stop and it keeps learning on its own. Every teacher reply is screened first (`_usable_lesson` rejects CLI-error strings and stubs) so the brain never distils *"Error: exceeded budget"* as if it were language — a small but load-bearing robustness guard on the core learning path.

Resonating in parallel. Because the spiking substrate batches over streams, the brain can run *k* thought-streams at once in a single forward pass — `SpikingBrain.resonate(k)` — for essentially the wall-time of one (6 streams measured at 9 ms, the same as one). Every few seconds the living loop resonates, keeps the stream it itself finds most fluent (lowest self-perplexity), and lets that reflection re-enter its mind — parallel exploration collapsing to a better single thought, and it learns from its own best thinking (§9). The teaching conversation with Claude Sonnet 5 runs through `brain/partner.py` (`claude_say` shells the `claude` CLI for a teacher paragraph; `web_topic` pulls clean Wikipedia extracts), verified to feed this same learning path end-to-end: a real lesson measurably lowers the cortex's bits/byte on that lesson. Every teacher reply is screened first (`_usable_lesson`) so a CLI error string is never distilled as language.

15.10 Memory hierarchy — RAM · SSD · eviction

`brain/memory.py` . The whole life of experienced text is tiered within fixed budgets: RAM holds the recent hot buffer; at the RAM/soft limit the oldest hot text is flushed and **compressed** to an SSD segment (zstd-19, ~100×); at the global/hard disk limit the **oldest compressed segments are deleted**. Sleep replays random chunks from hot and a decompressed old segment, so the brain rehearses its whole life. Text compresses enormously, so a long life fits in a small, bounded footprint. Model parameters are bounded by `max_model_gb` .

15.11 Device, precision, scale

`life.pick_device` . Auto CPU/GPU; **bf16 mixed precision (fp32 master weights, matmuls autocast to bf16) on GPU AND CPU** — bf16 *parameters* would wreck Adam's running averages, so the weights stay fp32 and only the compute casts. bf16 on CPU is honoured when asked for, with the honest caveat that without AVX512-BF16 it is *slower* than fp32 (emulated) — it is a GPU-throughput lever, not a CPU one. The checkpoint is device-agnostic (start on CPU, resume on GPU), and — new — **each of the five systems can run on its own device** (`set_part_device`

/ `POST /api/device`): the living loop converts tensors at every cross-region boundary, so a large cortex can sit on the GPU while a module stays on CPU. Multi-GPU FSDP2 sharding remains the documented next step.

Scaling to a large brain — the sparse cortex. A dense recurrent cortex is O(neurons²): at 500 000 neurons the recurrent matrix alone is ~750 GB — impossible. Above a threshold each cortical layer therefore switches to a **CSR sparse connectome** — int32 wiring buffers + a dense value `Parameter` + a 1-D active-synapse mask — so memory is O(neurons·fan-in) and the neuron count can reach the hundreds of thousands within RAM. The subtle part is *training*: `torch.sparse.mm`'s own backward transiently densifies a full neurons² gradient (measured ~4 GB/layer at 64 k, ~250 GB at 500 k → OOM). A custom autograd (`_SparseConnMM`) computes the value-gradient by a sampled gather/scatter instead — **O(nnz·B), no neurons² transient, verified bit-exact against the dense scatter** — so a half-million-neuron brain is genuinely *trainable*, not merely constructible. The small-net **dense path stays byte-identical** and is used below the threshold, so nothing about the default behaviour (or the test suite) changes. All four growable systems are sparse; the cortex is the only one that needs CSR (the modules are neurons×const, i.e. linear, so a masked dense store is already efficient). Honest ceiling: on CPU the recurrent per-timestep `sparse.mm` is poorly threaded, so at 64 k a learn step is tens of seconds — the sparse cortex is built for **GPU**, where that loop is fast; on CPU the trainable-in-real-time sweet spot stays a few-thousand-neuron dense net.

15.12 Checkpoint and resume a life

`life.save_life/load_life`. A life is checkpointed on night boundaries (not per step, which would stall at scale) plus a time fallback: weights + optimiser, and the life state — cycle, age, nights, clock, wake/sleep, mind. `--resume` continues exactly; the compressed episodic archive persists on disk.

15.13 Running it

- `python interface/dashboard.py` — the unified web board + HTTP API (recommended; §15.15). One URL: live charts, thought stream, log, vision, chat, status/config, and launch/stop/kill.
- `python run_life.py` — headless (same life; TensorBoard under `runs/<run>/tb`).
- `python interface/tui.py` — an opencode-style terminal interface. `--core spiking|rnn`, `--checkpoint <run>` to resume.
- `python test/run_tests.py` — the full self-contained test suite (every module + the full life + the dashboard API), 54 tests in seconds, no pytest required.

15.14 The five systems, actually coupled

An earlier version built the four non-cortex modules but never ran them — only the cortex lived. They are now wired into the loop, coupled by **activation and reward only, never gradients** (§6), at *zero* throughput cost (measured 1.00× with the modules on):

- **§2 basal ganglia → curiosity.** A spiking LIF **medium-spiny** actor-critic (growable — its neurons grow alongside the cortex) chooses *which topic* the brain asks Claude about next; its dopamine reward is the **learning progress** on the lesson — the drop in training loss, already computed during learning, so the reward is free. Over time the brain prefers the topics that teach its cortex the most. (Fixing the frozen-actor bug — the old `train_step` updated only the critic — was what made this possible; on a contextual bandit the fixed version learns the optimal policy, 0.33 → 0.98, while the old one stays at chance.)
- **§4 hippocampus → complementary learning systems.** Every experience is fingerprinted; its **novelty** (one minus the best recall match) both **gates whether it is written at all** (familiar patterns are not re-stored, so the buffer keeps the novel ones — encode without overwriting, §7.4) and sets how hard sleep replays it — a novel life consolidates harder. The hippocampus grows its dentate gyrus alongside the cortex each night. The store is a spiking modern-Hopfield associative memory: measured pattern-completion fidelity 1.00 → 0.96 as the number of stored memories goes 10 → 400, where the old covariance-Hopfield version collapsed and could not even reconstruct the pattern (it returned only a DG code) and had no `grow()` at all — a §10 growability violation now fixed.
- **§5 neuromodulation → plasticity gate.** The ACh/NE/DA/5HT tone is set on every wake↔sleep transition, and — this is the real coupling, not just a readout — the **ACh tone scales the cortex's effective learning rate** on every wake learning step (the three-factor gate M(t) in $\Delta w = \eta \cdot M(t) \cdot e$), so waking encodes at full plasticity and you can throttle it live via `/api/net`. (Only wake and NREM tones are entered; a REM sub-phase is named as roadmap, not claimed as running.)
- **§1 cerebellum → a fast supervised forward model.** Alongside the slow e-prop cortex it runs a *fast* supervised next-byte predictor, learned by its own climbing-fibre delta rule and gradient-cut from the cortex (§6): a bag-of-bytes window is the mossy input, a Golgi-controlled sparse granule expansion the hidden code, a Purkinje readout the prediction. Its error (`cerebellum_mse`) is tracked live. It complements the cortex (a fast vs slow learner, as in the cerebellar–cortical division of labour) and is a monitored forward model, not yet fed back into generation — the honest state of its integration.

On a homogeneous mocked feed the modules are within run-to-run noise on the cortex's own language metric (the mock ignores which topic was chosen, so curiosity cannot differentiate); their measured value is that they are now **free** (no regression) and **faithful** — all five are instantiated and running concurrently, coupled via reward (§2), tone/plasticity-gate (§5), novelty-gated replay (§4) and the cerebellar forward model (§1), where an earlier version had never even instantiated the cerebellum. (What is *not* yet built is the §6 router that would fuse the modules' competing

outputs by reliability — the cortex alone drives generation and the cerebellum's prediction is monitored, not fed back; that fusion is named as the next step.) Their real payoff shows on heterogeneous real-world input. Checkpoint/resume persists **all five modules' state and every live-tuned knob**, and a latent resume-after-growth bug (the loader assumed uniform layer width, but growth widens only the top layer) was found and fixed — resume is now byte-identical after arbitrary growth.

15.15 The cockpit — driving and extending the brain by API

A living thing you cannot watch or steer is a black box. The implementation therefore ships a single **web control-plane** (`dashboard.py`) that is also a complete HTTP API — *every* action the browser can take is one documented call (`GET /api/help` lists them), so the brain can be driven entirely programmatically. One command (`python interface/dashboard.py`) serves, on one URL: live metric charts, the timestamped stream of thought, the life log, the ASCII vision, a chat box, a full status/hyperparameter panel, and controls. On a remote host it prints the exact `ssh -L` tunnel to reach it. It supersedes the separate `tail -f` + TensorBoard + TUI (though TensorBoard still logs **~46 metrics** — capability, growth, wake/sleep, the five modules, curiosity, memory, throughput, weight-health — for deep, zoomable history).

- **Lifecycle & compute.** Launch, **graceful stop** (a STOP control file → it checkpoints and exits; no `kill -9`), or force-kill — from the board or the CLI. A run with no checkpoint gets a fresh timestamped folder (old runs untouched); `--checkpoint` continues one in place, and because the checkpoint is device-agnostic you can start on CPU, stop, and resume on GPU or multi-GPU. An **`auto`** compute mode maximises the hardware (every GPU, else all CPU cores); parallelism (CPU threads, the width *k* of parallel thought-resonance) is a knob. Resume skips re-birth — it is already competent.
- **Everything is live-tunable.** The design principle is that *any* parameter that can change without rebuilding the tensors changes **live**, no restart — the loop reads them each iteration, so a `POST /api/set` propagates on the next tick. That covers the global knobs (budget, wake/sleep windows, perceive-gap, think-chunk, learn-steps, resonance width, threads), the **development / cycle controls** (per-night growth amount, grow/prune ages, and hard toggles to *freeze growth*, *freeze the wake/sleep cycle* so it stays awake, or *freeze learning* so it observes without updating weights), and every **storage cap** below. Only the genuinely structural choices — initial neuron count and layer depth, device, core — need a `POST /api/start` relaunch (which resumes the checkpoint, so nothing is lost). Individual **parts of the net** are tuned through `POST /api/net` (per region, or `target:"all"` at once): the cortex's learning rate / readout mix / context length / sampling temperature, the hippocampus's recall sharpness / sparsity / capacity, the basal ganglia's actor & critic rates, the neuromodulator tones, and **each region's own synaptic growth/prune rate**. (On development: the *neuron count is fixed at birth* — a settable population — and it is the

synapses that evolve; you set the initial count, the initial synapse density, and per-region grow/prune rates, or freeze it. See §15.3.)

- **Architecture census & live surgery.** `GET /api/arch` reports, per region **and** globally, the neuron count, active-synapse count, parameter count, synapse density, per-layer widths, the per-region grow rate, and the region's device. `POST /api/arch` performs live surgery on any region (or `all`): grow neurons, **set** neurons to a target, grow/prune synapses, **set** synapses to a target count or density, or re-seed a connectome — so the living architecture is both fully observable and directly editable while it runs. `POST /api/device` places any region on its own CPU/GPU (tensors convert at the boundaries).
- **Resource budget, watched against its limits.** `GET /api/resources` (and time-series charts + a usage-bar panel) report RAM (this process **and** the box), VRAM per device, and every bounded store — checkpoint, replay, log, TensorBoard — as *current vs cap*, plus disk free/total. A single `GET /api/diag` is a one-call health check (alive? learning, with recent trends? all five systems firing? + explicit warnings), and `GET /api/value?key=<dotted.path>` fetches *any* value from the live snapshot — all built so the brain can be driven and diagnosed with no eyes on the dashboard at all.
- **Deeper diagnostics (state of the net).** Beyond capability, the board and TensorBoard track the *internal* state: perplexity on what it trains on **and** on what it generates, the entropy of its output distribution, the mean spiking rate (dead-neuron / saturation tripwire), per-layer weight magnitude and spread, the cerebellum's forward-model error, the basal-ganglia policy entropy, and the hippocampus's live recall fidelity — so you can see not just *whether* it is learning but *how healthy each of the five systems is* while it does. And because every live-tuned knob and each module's state is checkpointed, a resumed life continues exactly how you left it — same tuning, same growth schedule, same focus.
- **Bounded everything.** Like the replay buffer (§15.10), all on-disk artifacts stay within live-settable caps that evict the earliest data when full: the life-log truncates to its most recent lines, the TensorBoard event dir deletes its oldest files, the replay buffer its oldest segments, and the checkpoint is a single overwritten file bounded by the model-size cap — so a long experiment's footprint is fixed and known.
- **Directed teaching & attention.** `POST /api/teach` folds specific content in *now* — raw text, a wiki topic, a web page, or a local **file/dir of code, music notation, or prose** (it learns any byte stream, so its own source code is a valid lesson, verified). `POST /api/focus` redirects the learning feed from broad/random to a target area (topics, urls, mode), so you can say "learn music" or "learn coding" and the curiosity loop curates toward it.
- **Tools & other minds (the unified code, extended).** `POST /api/tools/add` registers any CLI tool or other AI — *opencode* with a free model, a local LLM, a text-to-speech or image command — as a template with an `{input}` slot and an output **modality**. The brain converses with it (autonomously or on demand), and its output is folded into the ONE byte stream by §15.6's encoders: text is learned as language; audio becomes cochlea bytes, an image retina bytes, anything else raw levels. This is the field guide's thesis made

operational — *any* sense or tool, or a combination, becomes "electricity" the brain learns from — and it is now extensible at runtime rather than fixed at build time. Each tool carries an optional install command (`POST /api/tools/install`) and enable / manual↔autonomous toggles (`POST /api/tools/toggle`); specs persist with the checkpoint.

- **Replaying what it sensed.** Non-text observations (an audio clip, an image a tool produced) are kept as their encoded byte frames and can be **reconstructed back into playable/viewable media** (`GET /api/observe`) — a deliberately low-fidelity rendering, because it decodes exactly the down-sampled, quantised "electricity" the brain actually received, not the original file. On the board you hear the cochlea bytes and see the retina bytes; in the thought feed each perception is tagged with its modality. It is the closest thing to looking through the brain's own senses.

The point is not the UI; it is that the whole living system — its rhythm, its diet, its senses, the minds it talks to — is inspectable and steerable while it runs, by a human at a glance or by another program through the API.

15.16 The honest ledger

What is real, on CPU: five different structures (§1–§5) — four growable spiking networks (cortex, cerebellum, hippocampus, and the basal-ganglia LIF medium-spiny actor-critic) plus a scalar neuromodulatory glue — coupled by the gradient cut and now **actually operating together** (§15.14), the neuromodulator ACh tone genuinely gating cortical plasticity; a spiking cortex that learns by DEFAULT by **faithful e-prop** (forward-in-time eligibility + a top-down learning signal + the three-factor gate; no BPTT, no weight transport — §15.17), with surrogate-BPTT (= $\beta \to 0$ PC) kept as an opt-in capability reference, reads its graded membrane, runs ~7× faster after a bit-identical vectorisation, and grows by §10 synaptogenesis with the function preserved; parallel thought resonance; a unified byte code for every sense; an internal clock; an autonomous, homeostatically-balanced, learning-during-sleep 24-hour loop with novelty-gated replay and dopamine-driven curiosity; continuous thinking with non-blocking human feedback; a compressed, evicting, whole-life memory in fixed RAM/SSD budgets; checkpoint/resume (byte-identical after growth); born-competent via teacher distillation, then learning from a verified Claude Sonnet 5 conversation; the full §10 developmental arc is real — a **fixed neuron population** with an **evolving synaptic connectome**: the child runs **synaptogenesis** (activating silent connections, densifying the wiring), the adolescent **prunes** the weakest synapses (mask-persistent), the adult stabilises — with deliberate neuron growth available on demand; and all five systems are genuinely instantiated and running, including the §1 cerebellum as a monitored fast forward model. The whole living system is watchable and drivable through one web board that is also a complete API (§15.15): launch/stop/resume across devices, live hyperparameter tuning, directed teaching, learning-feed steering, and runtime-registered tools / other AIs whose any-modality output folds into the same byte code. The chosen spiking route is faithful FIRST, so its fluency is

still modest — improved from ~4 to **~3.1–3.2 bits/byte** by the membrane readout and longer context, with real word-like fragments — with the rate-cortex reference (`--core rnn`, ~0.5 bits/byte, coherent sentences) one switch away when fluency is wanted. Every architecture choice here was settled by a controlled A/B, and the ones that *lost* (adaptive-threshold ALIF at short context, the spike/membrane mix, extra depth) are documented as such, not hidden. Neither route yet has long-range coherence, in-context learning, or reasoning — those exceed local credit assignment and are named, not oversold, as the next chapter: a deeper temporal spiking trunk and the two-compartment top-down path (which needs the slower time-outer schedule). The step §13 called last — instantiate the whole thing and watch it live — is taken here.

15.17 The faithfulness stack, and its measured price

The cortex no longer learns by backprop-in-a-costume. **e-prop is now the default rule** — a forward-in-time approximation to BPTT (Bellec et al. 2020): a per-synapse *eligibility trace* is kept online, a *top-down learning signal* gates it, and a *three-factor* neuromodulator term (the ACh tone, §15.14) opens or closes plasticity. There is no backward pass through time, no weight transport, no separate error network — the update is `@torch.no_grad`, computed by hand from local traces and applied with `w.add_`. Surrogate-BPTT + Adam is kept only as the opt-in, non-plausible **capability reference** (`learn_rule="bptt"`). The single non-local operation that remained (a global gradient-norm clip) is gone: the update is `Δw_ji = −η·M·clamp(mean_t[L_j·e_ji]/N_j, ±Δ)`, where each synapse sees only its own pre-trace, post learning-signal, and the diffuse M, and the per-postsynaptic-**fan-in** normalisation `N_j` (a homeostatic input-scaling) makes the stable rate **width-invariant** — verified identical descent at 8k↔64k↔256k neurons.

On top of that, **every biological constraint is an independent, live-toggleable axis** (extending the `learn_rule` switch — `set_faith(...)` / `POST /api/net`), so each one's cost can be *measured* rather than asserted:

- **Learned feedback (Kolen–Pollack)** vs. fixed-random DFA — the top-down matrix *learns* to align with the readout (its own local gradient + a decay), removing the random-alignment gap without weight transport. Metric: the feedback↔forward cosine.
- **Dale's law** — each neuron is excitatory or inhibitory; its outgoing synapses share one sign (~80/20 E:I), re-projected after every step. Shrinks the usable weight space.
- **Dendritic / burst error** — the learning signal is delivered as an apical **burst** that rides a somatic spike and is thresholded (Naud/Richards) — low-bandwidth, noisy, activity-coupled.
- **Unified two-compartment circuit** — the biological *completion* of the above (see below).
- **Bounded synapses** (Fusi) — weights clamped to ±w_max (real synapses are bounded, low-precision).
- **Firing-rate homeostasis** (metaplasticity) — each neuron's threshold drifts to hold a target rate (Turrigiano), keeping a continually-learning net off silence and saturation.
- **BTSP eligibility** (Bittner–Magee) — the eligibility trace outlives the membrane, widening the temporal-credit window beyond e-prop's ~10-step decay.

- **Differentiated neuromodulation** — the four tones gate four pathways (ACh→cortical encoding/the VIP drive, DA→reward-scaled plasticity, NE→somatic gain, 5-HT→apical patience), not one scalar dial.
- **Stochastic spiking** — probabilistic firing (membrane noise before threshold; noisy vesicle release).
- **Metabolic cost** — a spike-rate penalty in the learning signal (real coding is energy-constrained).

Self-adapting plasticity, and why the hyperparameters are scale-free. The effective learning rate is *not* a dial to hand-tune — it is `eprop_lr_scale · attention`, where **attention** self-regulates on the brain's own learning health: each step's loss is compared to a running baseline, and a loss spike above baseline (a shock, or over-plasticity) *drops* attention → the update shrinks → the representation is protected and re-learns gently (the Yerkes–Dodson arousal→plasticity curve). This makes learning self-healing — an injected weight shock recovers autonomously, and the rate that once caused a runaway stays bounded — and it removes hand-tuning. Sleep, correspondingly, cycles NREM↔REM under the §5 tones with replay depth gated by the day's novelty/debt, so *when* and *how hard* it consolidates is set by the day, not a constant. The design is **scale-invariant by construction**, which is what lets the same hyperparameters move from a 16k to a 256k to a million-neuron cortex without retuning: the base rate is **fan-in-normalized** (÷N_j → the per-neuron drive change, and hence the representation magnitude, is width-invariant — verified: `mem_mag` 1.357 at 16k vs 1.367 at 64k under identical settings), attention runs on the *relative* loss (dimensionless → size-independent), and the Δmax bound is per-synapse. A suite of leading-indicator diagnostics (`mem_mag` = representation magnitude, the true runaway signal that climbs *before* bits/byte does; `update_mag` , `grad_mag` , `attention` , `eff_lr_scale` , `surprise` , `loss_ema`) is exposed in `GET /api/state` , because bits/byte alone lags the dynamics.

One circuit, not separate toggles (the unification). The toggles above are *axes*, but three of them are really one biological circuit, and `two_compartment=True` fuses them: each neuron gets an **apical dendrite** (the §3.7 `TwoCompartmentLIF` compartment, now load-bearing) with its own membrane `ap ← β_ap·ap + gate·(err·B)` ; the top-down error is **admitted only through a VIP→SOM disinhibition gate** — SOM inhibition rises with local activity, and VIP, driven by the neuromodulator "learn-now" tone `M` , disinhibits it (`gate = (VIP − SOM)₊`); the apical membrane then **bursts onto somatic spikes** to drive plasticity and **feeds back onto somatic firing** (`drive += g_ap·ap`), with PV supplying fast divisive gain control. So the *substrate* (apical compartment), the *error delivery* (into it), the *interneurons* (SOM/VIP gate), and the *neuromodulator* (drives VIP) stop being four separate features and become **one microcircuit** — the error runs *through* the apical dendrite, not alongside it. This subsumes the standalone `dendritic` toggle (kept for the not-yet-routed comparison).

The measured capability–fidelity curve. Training an otherwise-identical 16k-neuron cortex under each constraint (added one at a time, identical seed/data, 220 steps) gives the cost of each

biological commitment — the thing the field usually hand-waves about:

Configuration	bits/byte	cost vs. plausible base
BPTT (non-plausible ceiling)	2.42	−3.78
e-prop + random feedback (DFA)	6.21	+0.01
+ learned feedback (Kolen–Pollack)	6.20	0.00
+ Dale's law	5.58	−0.62
+ dendritic / burst error (standalone)	6.58	+0.38
+ bounded synapses	6.20	0.00
+ firing-rate homeostasis	6.21	+0.01
+ BTSP long eligibility	6.06	−0.14
+ **unified two-compartment** (apical/SOM-VIP/neuromod)	5.90	−0.30
+ differentiated neuromodulation (4 tones → 4 pathways)	5.96	−0.24
+ stochastic spiking	6.19	−0.01
+ metabolic cost	6.34	+0.14
full faithful stack (all mechanisms)	**5.10**	**−1.10**

The honest reading: **the dominant price of plausibility is the learning *rule*** — e-prop costs ~3.8 bits/byte against BPTT — while the individual biological *constraints* are mostly near-neutral or even *helpful* at this scale. Three results stand out. First, **routing the error through the apical compartment beats bolting it on**: the unified `two_compartment` circuit (5.90) is far better than the standalone `dendritic` burst (6.58) — the SOM/VIP-gated apical integration is not only more faithful but more capable than a thresholded side-signal. Second, **differentiating the neuromodulator helps** (−0.24 in isolation): four tones gating four pathways (ACh→encoding, DA→reward-scaling, NE→gain, 5-HT→patience) out-learn one scalar dial, and it couples learning to the wake/sleep cycle (verified: NREM tones down-modulate plasticity vs. wake). Third, **the constraints co-operate rather than compound**: the full stack (5.10) *beats* plain e-prop+learned-feedback (6.20) by ~1.1 bits/byte — the genuinely unexplored result, since stacking that many approximations usually makes a net learn far worse or not at all. Two findings to hold honestly. **Dale's law shows a small *improvement*, not the expected cost** — consistent in sign across seeds (−0.12 to −0.14 at 16k, both seeds; −0.6 in the headline config), a real regularisation effect rather than noise, though its magnitude is config-dependent and it raises the spike rate (so we leave it off the default live run for stability). And **the metabolic penalty correctly *costs* capability** (+0.14) while pulling the spike rate down (0.040→0.033) — the honest energy↔capability trade, and the sign that a first implementation got backwards (a spike penalty must *raise* the per-neuron error, not lower it). (Reproduce:

`runs/fidelity_capability_curve.py`; the numbers refresh into `runs/fidelity_capability_curve.{json,md}`.)

Where this sits, and the permanent gap. On the *learning rule* this is frontier-grade — few groups run e-prop + learned feedback + Dale + dendritic/burst + three-factor neuromod + homeostasis in one spiking net that learns language-like bytes continually. But "faithful to how the brain learns" is roughly a dozen independent axes, and the true cortical algorithm is *unknown* — there is no confirmed single algorithm (predictive coding, the NGRAD/backprop-approximation family, and "prospective configuration"/equilibrium-style relaxation are live, partly-incompatible hypotheses; the strongest phenomenon-level evidence is for prediction-error responses gated by SST/VIP interneurons — Furutachi/Mrsic-Flogel/Hofer, *Nature* 2024 — but that does not uniquely confirm any one algorithm). So the honest status is: frontier on the rule, **mid-field on the other axes, and provably short of "solved," because solved is not yet knowable by anyone.**

The roadmap (named, not oversold). Ranked by whether they genuinely change *learning*:

- *Group A — changes learning (built):* the **PV/SOM/VIP-gated apical circuit** (unified `two_compartment`) and **differentiated four-tone neuromodulation** are now in — the connective tissue between dendritic error and neuromodulation, and the four-pathway modulator that plugs into it. What remains in this tier: **real interneuron populations** — PV/SOM/VIP are currently *functional mean-field* terms (`som = som_b·⟨z⟩`, `vip = ACh`, PV = divide-by-mean), not separate spiking LIF populations with their own connectomes and Dale typing; making them explicit populations is the faithful version. **STDP** (a millisecond spike-timing kernel for the eligibility, not a rate low-pass). **Cascade/complex synapses** (Fusi) for catastrophic-forgetting resistance beyond a bound. A short **relaxation / prospective-configuration** micro-phase (settle activity under top-down nudging before the update) — the biggest single lever per the literature (Song et al., *Nat. Neurosci.* 2024), complementing e-prop's temporal traces with spatial relaxation, and a knob to test a predictive-coding objective against the current global readout.
- *Group B — when it learns:* **sharp-wave-ripple / theta-gamma gating** of replay (we schedule replay by a debt heuristic; biology gates it on oscillation events); **adult dentate-gyrus neurogenesis** (the one place automatic neuron growth is the faithful choice).
- *Group C — realism, steeper diminishing returns:* richer dendrites (NMDA plateaus), stochastic spiking + a metabolic/energy term, short-term plasticity.
- *Group D — the setup, not the synapse:* a closed **sensorimotor / active-inference loop** where the brain's outputs contingently reshape its future input. This is arguably the deepest gap in how the brain learns — biological learning is grounded in an action–perception loop with consequences — and it is about the *setup*, not the rule. Our brain reads and talks to Claude, but its words do not yet change its world.

Faithfulness is **asymptotic**: the brain has effectively unbounded biophysical detail, so there is always another axis. The research value is not checking every box — it is (1) separating the axes that change learning (group A) from the realism that does not (group C), and (2) *measuring the*

fidelity-vs-capability curve as each is added. That curve — "here is exactly how much each biological constraint costs" — is the contribution; the full survey lives in `runs/cortical_algorithm_research.md`.

§16 · The deeper brain: one neuro-endocrine-oscillatory control loop (R&D)

§15 gives five spiking modules + a self-adapting cortex, but every cycle fires all five and sleep replays a **raw byte buffer** — two things the brain almost certainly does not do. Three literature-grounded R&D threads (full surveys + citations in `runs/{memory_architecture,drive_stress,dynamics_oscillations}_research.md`; integrated design in `runs/deeper_brain_integrated_design.md`) converge on a single missing controller that sits *above* the existing tones, the self-adapting `attention`, the sleep-debt, and the dopamine critic — coupling only to fields that already exist:

> **endocrine drive/cortisol** (sets the arousal/entropy operating point + **sleep pressure**) → the **ultradian sleep-cycle** (NREM/REM schedule) → **ripple-gated generative consolidation** of the **generalized-in-the-net** memory → gated by **oscillatory dynamic states** (which regions fire, at what frequency, set by attention) → feeding back into the self-adapting attention.

Memory is generalized-in-the-net, not a buffer (verified). The raw replay buffer is biologically wrong: hippocampal replay is *reconstructive/generative* — it builds never-experienced shortcuts (Gupta 2010), pre-plays untraversed paths (Ólafsdóttir 2015), and sweeps to *future* goals (Pfeiffer-Foster 2013); under Complementary Learning Systems (McClelland 1995; Kumaran 2016) the cortex accumulates *generalized* structure in its weights, nothing stores raw episodes. The buffer-free mechanism is **generative self-replay** (Robins 1995; Shin 2017; van de Ven 2020): the cortex *dreams* from its own dynamics and hard-learns the dreams. **This is built and measured** — `SpikingBrain.generative_replay()` (diverse high-temperature dreams from sparse byte-cues, with two anti-"overfitted-brain" safeguards: a small veridical anchor fraction and a held-out acceptance monitor). On a forgetting-resistance test (learn topic A, then live through B–E), topic-A retention was **none 0.232 < raw-buffer 0.255 < generative 0.274** — dreaming from the net consolidates *better* than replaying stored text, with **no buffer** (`runs/generative_replay_test.py`).

The drive/stress layer. A `SpikingEndocrine` of slow scalars — a **drive deficit** (leaky integrator, met by a satiation event → a homeostatic-RL reward, Keramati-Gutkin, fed into the existing dopamine critic), **cortisol** (rises with unmet drive + prediction-error; an inverted-U on plasticity — acute sharpens, chronic impairs; and it *is* the sleep-pressure term), and **mood** (a dopamine EMA that damps cortisol — resilience). "Satiation → focus" is principled: a satisfied drive lowers tonic NE → the phasic, focused mode (Aston-Jones- Cohen). (Energy is modelled as an opportunity-cost bias, not a fuel gate — the glucose/ego-depletion account is discredited, Hagger 2016.)

The dynamics layer. Not-all-regions-active is an **ignition gate** (global workspace, Dehaene) — each system ignites only if its salience beats a threshold set by a single **entropy knob β** (the normal↔psychedelic axis, REBUS/Carhart-Harris). Attention shifts the **processing frequency** (gamma-fast window when focused, alpha-slow when disengaged, theta sampling) — a variable effective eligibility-window over the fixed tick. Sleep becomes a fixed **ultradian FSM** (SWS-heavy early → REM-heavy late) with SO-spindle-ripple coupling gating *when* consolidation commits.

Honest status + phased plan. An adversarial pass (in the design doc) confirmed the *control* couplings are one real loop but flagged that the current histogram-based hippocampus cannot seed a semantic "gist" — hence P0 uses sparse *byte-cues*, not vector reinstatement. Build order by value-per-effort: **P0 generative self-replay (done + verified)**; **P1** `SpikingEndocrine` over the existing knobs (drive→reward→focus, cortisol inverted-U, sleep-pressure); **P2** the oscillatory ignition/frequency layer; **P3** the full SO-spindle-ripple sleep FSM + a richer hippocampal trace so reinstatement can carry sequence, not just letter statistics. Each ships behind a toggle with an acceptance test, and lands one piece at a time — the lesson of the faithfulness build.